Quiz

COMP102: Data Structures

University Mohammed VI Polytechnic — SASE

Academic Year 2025–2026

Instructions. Answer all questions on a separate sheet. For classification questions, choose all answers that apply. For tracing questions, show the state after each operation. For coding questions, write **valid** Python code. Total: **20 points**.

Question 1 — 1 points

Explain the difference between an **Abstract Data Type** and a **Data Structure**. Give one example where the same ADT can be implemented using different data structures.

Question 2 — 1.5 points

Consider the following Python values:

```
17
"data"
(4, 9)
[4, 9]
{"x": 1}
(2, (3, 4))
(2, [3, 4])
```

Which of these values can safely be inserted into a `set` or used as keys in a `dict`?
For each value, briefly explain your decision.

Question 3 — 1.5 points

What is printed by the following code?

```
a = [2, 4]
b = a
c = [a, b]
a.append(6)
b = b + [8]
c[1].append(10)
print(a)
print(b)
print(c)
```

Question 4 — 2 points

For each situation, choose the best data structure or ADT and justify your choice in one sentence.

1. You need to test quickly whether a username has already been used.
2. You need to store students in the exact order they registered and access the 10th student directly.
3. You need to map each student ID to the student's grade.
4. You need to repeatedly undo the most recent editing operation.
5. You need to process customers in the order they arrived.
6. You need frequent insertion and deletion at the beginning of a sequence.
7. You are given a direct reference to a node in the middle of a sequence and must insert a new item immediately after it in constant time.
8. You need to repeatedly extract the task with the smallest priority number.
9. You need to maintain values where search, insertion, and sorted traversal are all important.
10. You need to count how many times each word appears in a text.
11. You need to simulate function calls and returns.

Question 5 — 2 points

For each binary tree below, choose **all** classifications that apply:

BST AVL tree Min-heap Max-heap None of these

Use the following conventions:

- A BST uses the strict rule: left subtree values $<$ root $<$ right subtree values.
- An AVL tree is a BST where every node has balance factor -1, 0, or 1.
- A binary heap must satisfy the complete-tree shape property and the appropriate heap-order rule.

Tree A

```
        8
       / \
      3   10
     / \    \
    1   6    14
```

Tree B

```
        1
       / \
      3   2
     / \ / \
    7  4 5  6
```

Tree C

```
        9
       / \
      7   8
     / \ / \
    1  3 4  2
```

Tree D

```
        10
       /  \
      5    15
       \
        12
```

Tree E

```
      4
     /
    3
```

For each tree, briefly justify your classification.

Question 6 — 1.5 points

A stack starts empty. Represent the stack **vertically**. Trace the stack after each operation.

```
push(3)
push(7)
push(2)
pop()
push(9)
peek()
push(4)
pop()
pop()
push(8)
push(1)
pop()
```

For each step, write the operation, the returned value if any, and the full stack after the operation.

Question 7 — 1.5 points

A queue starts empty. The front of the queue is shown on the **left**. Trace the queue after each operation.

```
enqueue("A")
enqueue("B")
enqueue("C")
dequeue()
enqueue("D")
front()
dequeue()
enqueue("E")
enqueue("F")
dequeue()
dequeue()
front()
```

For each step, write the operation, the returned value if any, and the full queue after the operation.

Question 8 **2.5 points**

Insert the following values into an empty AVL tree:

`50, 20, 70, 10, 30, 25, 27`

After each insertion:

1. Draw the tree.
2. Write the height of each node.
3. Write the balance factor of the first unbalanced node, if any.
4. Identify the AVL case: LL, RR, LR, or RL.
5. State which rotation or rotations are applied.

Draw the final AVL tree after all insertions.

Question 9 **2 points**

Start with an empty min-heap. Trace the heap array after each operation.

```
push(12)
push(5)
push(20)
push(3)
push(8)
push(15)
pop()
push(2)
pop()
push(7)
```

For each step, write the operation, the returned value if any, and the heap array after repair. Use zero-based heap indexing.

Question 10 **2 points**

Implement a class `MyStack`.
Required attribute:

`items`

Required methods:

`__init__` `push` `pop`
`peek` `is_empty` `size`

Rules:

- Use a Python list internally.
- `push` adds `value` to the top of the stack.
- `pop` removes and returns the top value.
- `peek` returns the top value without removing it.
- `is_empty` returns `True` if the stack has no elements and `False` otherwise.
- `size` returns the number of elements in the stack.
- Calling `pop` or `peek` on an empty stack must raise `IndexError("empty stack")`.

Question 11 **2.5 points**

Implement the function `count_digit(n, d)`.
The function returns how many times digit `d` appears in the nonnegative integer `n`.
Rules:

- `n` is a nonnegative integer.
- `d` is an integer from `0` to `9`.
- Solve the problem recursively.
- Do not convert `n` to a string.
- Do not use loops.

Examples:

```
count_digit(5050, 5)      # 2
count_digit(5050, 0)      # 2
count_digit(12345, 9)     # 0
count_digit(0, 0)         # 1
count_digit(0, 7)         # 0
```

After the code, explain the base case, the smaller recursive subproblem, how the current digit is processed, why the recursion eventually stops, the time complexity, and the recursion-stack space complexity.